

Dynamics and Stability of the Spring-Loaded Inverted Pendulum (SLIP) Model

Arael Anaya

1 Introduction and Relevance

The spring-loaded inverted pendulum (SLIP) is the reduced-order model behind most legged-robot dynamics [3, 5]: a point mass bouncing on a massless spring leg. Despite its simplicity, it captures the center-of-mass dynamics of running in both animals and machines remarkably well [1, 2]. That makes it a good subject for this course: it is simple enough to analyze fully with the tools covered here, but rich enough to produce genuinely interesting hybrid dynamics. It also builds directly on the elastic pendulum studied in class, since the stance phase of SLIP is an inverted spring pendulum.

What makes SLIP especially interesting from a mechanics standpoint is that it is a hybrid dynamical system: during stance the body behaves like a spring-loaded pendulum, while in flight it is a free projectile, with the two phases connected by touchdown and liftoff switching conditions. Rather than analyzing stability at a fixed equilibrium point, the more natural question for a hopping gait is whether the periodic motion itself is stable. The standard tool for that is the apex return map, a Poincaré map in which a steady hopping gait appears as a fixed point whose stability can be assessed numerically [1, 4].

Beyond being a convenient classroom system, SLIP-type models are used directly in the design and control of running robots and in the biomechanical study of animal and human locomotion, which is the practical motivation for this project: a model simple enough to analyze by hand but predictive enough to inform how a real legged system should be built or controlled.

Relation to Prior Work

This project sits within an established line of SLIP research, and each stage of the model draws on a different piece of that literature. Blickhan [1] introduced the spring-mass template and showed that it reproduces the ground-reaction-force and center-of-mass patterns of human and animal running; the stance-phase Lagrangian derived here reproduces that same reduced-order model, but from Lagrange's equations in polar coordinates rather than Blickhan's Newtonian force balance. McMahon and Cheng [2] used the same template to relate leg stiffness to running speed empirically; the parameter sweep over stiffness k and touchdown angle θ_{TD} planned for this project asks the same question, but numerically, from the model directly, rather than from experimental data. Full and Koditschek's templates-and-anchors framework [3] is the justification for treating SLIP as a legitimate object of study on its own, rather than only as a shortcut on the way to a fully articulated, multi-link leg model: a template is expected to capture the essential dynamics of the anchored system while discarding detail that does not matter for stability. Schwind and Koditschek [4] derived an approximate, closed-form stance map for SLIP using a perturbative solution; this project instead integrates the stance-phase equations numerically with an event-based solver, trading their analytical insight for an exact (if less interpretable) stance map, consistent with the computational tools emphasized in this course. Saranli, Buehler, and Koditschek [5] built RHex, a hexapod whose individual legs behave like a SLIP template during stance, which is the practical grounding for why this abstraction is worth studying: it is not just a toy system, it is the model actually used to understand a working robot. Finally, Geyer, Seyfarth, and Blickhan [6] extended a SLIP-type compliant-leg model to walking, not just running, by allowing double support; that extension is a natural direction for future work beyond the single-support hopping model considered here.

Expand this comparison if needed to more explicitly satisfy the “at least 5 scholarly articles” requirement with direct quantitative comparisons (e.g., comparing this project’s stiffness sweep results directly against McMahon & Cheng’s reported stiffness–speed trend, or against Schwind & Koditschek’s closed-form stance-map approximation) once the parameter sweep in Section 4 is complete.

2 Modeling of Dynamics

The equations of motion for both phases of the SLIP model are derived using **Lagrange’s equations of motion**, with generalized coordinates chosen separately for each phase: Cartesian coordinates (x, y) for the flight phase, and polar coordinates (r, θ) about the planted foot for the stance phase. The two phases are connected by touchdown and liftoff switching (transversality) conditions, so the overall model is a hybrid dynamical system integrated with event-based numerical methods. Figure 1 shows the geometry of both phases.

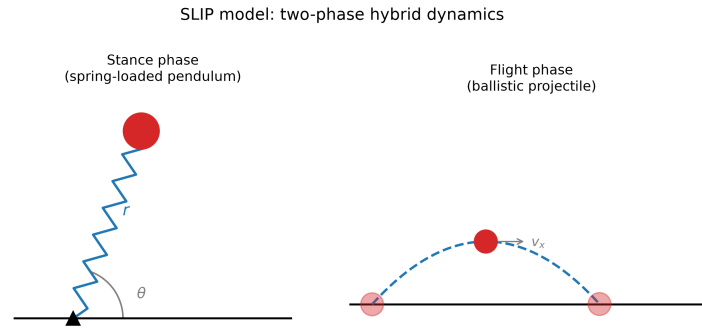


Figure 1: SLIP model geometry: stance (spring pendulum, r, θ) and flight (projectile).

2.1 Flight-Phase Equations of Motion

With basis $\hat{n}_1, \hat{n}_2$, $\vec{r}_m = x\hat{n}_1 + y\hat{n}_2$:

$$T = \frac{1}{2}m(\dot{x}^2 + \dot{y}^2), \quad V = mgy, \quad L = T - V = \frac{1}{2}m(\dot{x}^2 + \dot{y}^2) - mgy$$

Lagrange’s equations give

$$\ddot{x} = 0, \quad \ddot{y} = -g$$

Starting at the apex ($\dot{y} = 0$), with $x(0) = x_0$, $\dot{x}(0) = v_{x0}$:

$$x(t) = v_{x0}t + x_0, \quad y(t) = y_0 - \frac{g}{2}t^2$$

2.2 Touchdown: Detection and Transition Map

θ_{TD} is held fixed in flight, so the foot trails the body at

$$\vec{r}_{foot}(t) = (x(t) - r_0 \cos \theta_{TD})\hat{n}_1 + (y(t) - r_0 \sin \theta_{TD})\hat{n}_2$$

Touchdown is $y_{foot} = 0$, i.e. $y_{TD} = r_0 \sin \theta_{TD}$. Solving $y(t) = y_0 - \frac{g}{2}t^2$:

$$t_{TD} = \sqrt{\frac{2(y_0 - r_0 \sin \theta_{TD})}{g}}, \quad \dot{y}_{TD} = -\sqrt{2g(y_0 - r_0 \sin \theta_{TD})}, \quad x_{TD} = v_{x0}t_{TD}, \quad \dot{x}_{TD} = v_{x0}$$

Stance uses polar coordinates about the planted foot, so this Cartesian state must convert. With foot at $\vec{r}_F = (x_{TD} + r_0 \cos \theta_{TD})\hat{n}_1$ and leg vector $\vec{\rho} = \vec{r}_m - \vec{r}_F = -r_0 \cos \theta_{TD} \hat{n}_1 + y_{TD} \hat{n}_2$:

$$r = |\vec{\rho}| = \sqrt{r_0^2 \cos^2 \theta_{TD} + y_{TD}^2} = r_0, \quad \theta = \text{atan2}(y_{TD}, -r_0 \cos \theta_{TD})$$

Projecting $\vec{v} = \dot{x}_{TD} \hat{n}_1 + \dot{y}_{TD} \hat{n}_2$ onto $\hat{e}_r = \cos \theta \hat{n}_1 + \sin \theta \hat{n}_2$, $\hat{e}_\theta = -\sin \theta \hat{n}_1 + \cos \theta \hat{n}_2$:

$$\dot{r} = \vec{v} \cdot \hat{e}_r = \dot{x}_{TD} \cos \theta + \dot{y}_{TD} \sin \theta, \quad \dot{\theta} = \frac{\vec{v} \cdot \hat{e}_\theta}{r_0} = \frac{-\dot{x}_{TD} \sin \theta + \dot{y}_{TD} \cos \theta}{r_0}$$

implemented as `touchdownState()`.

2.3 Stance-Phase Equations of Motion

In stance the leg pivots about the fixed foot: $\vec{r}_m = r\hat{e}_r$, $\vec{v}_m = \dot{r}\hat{e}_r + r\dot{\theta}\hat{e}_\theta$:

$$T = \frac{1}{2}m\dot{r}^2 + \frac{1}{2}mr^2\dot{\theta}^2, \quad V = mgr \sin \theta + \frac{1}{2}k(r - r_0)^2, \quad L = T - V$$

Lagrange's equations:

$$m\ddot{r} - mr\dot{\theta}^2 + mg \sin \theta + k(r - r_0) = 0, \quad mr^2\ddot{\theta} + 2mr\dot{r}\dot{\theta} + mgr \cos \theta = 0$$

solved for the highest derivatives:

$$\ddot{r} = r\dot{\theta}^2 - g \sin \theta - \frac{k}{m}(r - r_0), \quad \ddot{\theta} = -\frac{2}{r}\dot{r}\dot{\theta} - \frac{g}{r} \cos \theta$$

implemented as `stance_RHS()`.

2.4 Conserved Energy in Stance

$\vec{r}_m = r\hat{e}_r$ is scleronomic and T is homogeneous quadratic in $(\dot{r}, \dot{\theta})$, so $H = T + V$; with no explicit t -dependence, energy is conserved through stance:

$$E = \frac{1}{2}m(\dot{r}^2 + r^2\dot{\theta}^2) + mgr \sin \theta + \frac{1}{2}k(r - r_0)^2 = \text{const.}$$

Neither coordinate is cyclic, so both equations must be integrated together; this conserved quantity is used below as a check on the numerical integrator.

2.5 Liftoff Map and the Apex Return Map

Liftoff occurs when the leg returns to its natural length with positive rate, $r = r_0$, $\dot{r} > 0$, at which point the stance-phase state $(r_0, \theta_{LO}, \dot{r}_{LO}, \dot{\theta}_{LO})$ must be converted back into Cartesian flight variables to begin

the next flight phase. Chaining flight $\rightarrow$ touchdown $\rightarrow$ stance $\rightarrow$ liftoff $\rightarrow$ flight once defines a map from one apex state to the next, $P : (y_n, v_{x,n}) \mapsto (y_{n+1}, v_{x,n+1})$, whose fixed points correspond to steady periodic hopping and whose local stability is governed by the linearization of P about that fixed point. Section 2.7 shows that this apparently two-dimensional map is in fact governed by a single scalar, so that linearization turns out to be one derivative, not a Jacobian's worth of eigenvalues.

Liftoff occurs at $r = r_0$ (the switching condition of Section 2.3), so the liftoff state is $(r_0, \theta_{LO}, \dot{r}_{LO}, \dot{\theta}_{LO})$. Position converts back to Cartesian with the same relation used throughout stance, $\vec{r}_m = r\hat{e}_r$ (Section 2.3), evaluated at $r = r_0, \theta = \theta_{LO}$:

$$x_{LO} = r_0 \cos \theta_{LO}, \quad y_{LO} = r_0 \sin \theta_{LO}$$

Velocity converts by substituting $\hat{e}_r = \cos \theta \hat{n}_1 + \sin \theta \hat{n}_2$, $\hat{e}_\theta = -\sin \theta \hat{n}_1 + \cos \theta \hat{n}_2$ (Section 2.2) into $\vec{v}_m = \dot{r}\hat{e}_r + r\dot{\theta}\hat{e}_\theta$ (Section 2.3):

$$\vec{v}_m = \dot{r}(\cos \theta \hat{n}_1 + \sin \theta \hat{n}_2) + r\dot{\theta}(-\sin \theta \hat{n}_1 + \cos \theta \hat{n}_2) = (\dot{r} \cos \theta - r\dot{\theta} \sin \theta) \hat{n}_1 + (\dot{r} \sin \theta + r\dot{\theta} \cos \theta) \hat{n}_2$$

evaluated at liftoff ($r = r_0, \theta = \theta_{LO}, \dot{r} = \dot{r}_{LO}, \dot{\theta} = \dot{\theta}_{LO}$):

$$\dot{x}_{LO} = \dot{r}_{LO} \cos \theta_{LO} - r_0 \dot{\theta}_{LO} \sin \theta_{LO}, \quad \dot{y}_{LO} = \dot{r}_{LO} \sin \theta_{LO} + r_0 \dot{\theta}_{LO} \cos \theta_{LO}$$

Note that this uses $\dot{\theta}_{LO}$, not just $\dot{r}_{LO}$: leg rotation rate contributes to Cartesian velocity just as much as leg extension rate does, through the $r\dot{\theta}\hat{e}_\theta$ term of $\vec{v}_m$.

Once airborne, r and θ have no physical meaning to condition on: they are polar coordinates about the planted foot, and there is no foot left to be “about” after liftoff releases it. Apex is instead defined purely in the ground frame, by $\dot{y} = 0$. Flight has $\ddot{y} = -g$ (Section 2.1), so, starting the clock at liftoff,

$$\dot{y}(t) = \dot{y}_{LO} - gt$$

Setting $\dot{y}(t_{apex}) = 0$:

$$t_{apex} = \frac{\dot{y}_{LO}}{g}$$

This is linear in t because it is a condition on velocity, one integration of the constant $-g$ away from $t = 0$. Touchdown (Section 2.2) instead solves a condition on position, $y(t_{TD}) = y_{TD}$, and $y(t) = y_0 - \frac{g}{2}t^2$ is quadratic in t (two integrations of $-g$) – which is why that closed form needed a square root and this one does not.

Propagating $x, y, \dot{x}, \dot{y}$ forward from liftoff using the flight solution of Section 2.1, $x(t) = \dot{x}_{LO}t + x_{LO}$, $y(t) = y_{LO} + \dot{y}_{LO}t - \frac{g}{2}t^2$, $\dot{x}(t) = \dot{x}_{LO}$, $\dot{y}(t) = \dot{y}_{LO} - gt$, at $t = t_{apex} = \dot{y}_{LO}/g$:

$$x_{n+1} = x_{LO} + \dot{x}_{LO} t_{apex}, \quad y_{n+1} = y_{LO} + \dot{y}_{LO} t_{apex} - \frac{g}{2} t_{apex}^2, \quad v_{x,n+1} = \dot{x}_{LO}, \quad \dot{y}_{n+1} = 0$$

$v_{x,n+1} = \dot{x}_{LO}$ exactly, since horizontal motion is unforced in flight ($\ddot{x} = 0$, Section 2.1); $\dot{y}_{n+1} = 0$ exactly, by construction of t_{apex} , not approximately.

The returned apex state $(x_{n+1}, y_{n+1}, v_{x,n+1}, \dot{y}_{n+1})$ is expressed in Cartesian coordinates, not polar. This is a deliberate representational choice, not an oversight: polar (r, θ) was only ever meaningful about the specific planted foot of the stance phase just finished, and once airborne there is no foot location left for the next flight segment to be polar about. (x_{LO} above is likewise expressed relative to that same foot rather than to some fixed ground origin; Section 2.7 shows this choice of origin cannot matter, since x never enters the dynamics of either phase.)

2.6 Conserved Energy at Apex and Across a Full Hop Cycle

The flight-phase Lagrangian (Section 2.1) has $\vec{r}_m = x\hat{n}_1 + y\hat{n}_2$ scleronomic and $T = \frac{1}{2}m(\dot{x}^2 + \dot{y}^2)$ homogeneous quadratic in $(\dot{x}, \dot{y})$, with no explicit t -dependence – the same structural conditions used in Section 2.4 for stance. By the identical argument, $H = T + V$ is conserved through flight:

$$E = \frac{1}{2}m(\dot{x}^2 + \dot{y}^2) + mgy = \text{const.}$$

with no spring term, since the leg carries no elastic potential energy while airborne (unloaded, uncompressed). This is the apex-state energy functional. It differs from the stance-phase functional of Section 2.4,

$$E_{\text{stance}} = \frac{1}{2}m(\dot{r}^2 + r^2\dot{\theta}^2) + mgr \sin \theta + \frac{1}{2}k(r - r_0)^2,$$

by the missing $\frac{1}{2}k(r - r_0)^2$ term. The two functionals coincide exactly at touchdown and at liftoff, because both switching events occur at $r = r_0$ (Sections 2.2, 2.3), where the spring term vanishes identically, $\frac{1}{2}k(r_0 - r_0)^2 = 0$. Total mechanical energy is therefore continuous across every touchdown and liftoff transition, and, since it is separately conserved within each phase, it is conserved across an entire cycle apex $\rightarrow$ touchdown $\rightarrow$ stance $\rightarrow$ liftoff $\rightarrow$ apex. Section 4.2 validates this numerically.

2.7 Dimension Reduction of the Apex State

At apex, $\dot{y} = 0$ by definition (Section 2.5). Substituting into the apex energy functional of Section 2.6:

$$E = \frac{1}{2}m\dot{x}^2 + mgy$$

Solving for $\dot{x}$:

$$E - mgy = \frac{1}{2}m\dot{x}^2 \implies \dot{x}^2 = \frac{2(E - mgy)}{m} = \frac{2E}{m} - 2gy$$

$$\dot{x} = \sqrt{\frac{2E}{m} - 2gy} \quad (\dot{x} > 0 \text{ for forward hopping})$$

The apex state $(x, y, \dot{x}, \dot{y})$ is therefore not four independent numbers. $\dot{y} = 0$ identically at every apex, by definition. x never appears in any potential or force in either phase (Sections 2.1, 2.3): it is an ignorable coordinate, free to translate rigidly from hop to hop without feeding back into $\dot{x}$, y , or $\dot{y}$ – which is exactly why Section 2.5 was free to express x_{LO} relative to the current stance foot rather than to a fixed ground origin. And $\dot{x}$ is not independently free: once E is fixed, it is determined by y through the relation above. So at fixed E , the entire apex state collapses to the single scalar y , and the hop-to-hop dynamics reduce to a one-dimensional map,

$$y_{n+1} = P(y_n)$$

where P is the composition: reconstruct the full apex state from y_n via $\dot{x}_n = \sqrt{2E/m - 2gy_n}$ (this section) $\rightarrow$ flight to touchdown (Section 2.2) $\rightarrow$ stance integration to liftoff (Section 2.3) $\rightarrow$ flight to the next apex (Section 2.5).

This reduction was checked numerically, not just asserted algebraically: the closed-form $\dot{x}$ above, evaluated at the *output* apex height y_{n+1} produced by simulating one full hop, was compared against the $\dot{x}_{n+1}$ that hop actually produced. Section 4.2 reports the result: the two agree to machine precision.

2.8 The Apex Return Map: Fixed Point and Stability Criterion

With the reduction of Section 2.7, define the apex return map P at fixed leg stiffness k and fixed touchdown angle θ_{TD} , with total energy E also held fixed:

$$P : y_n \mapsto y_{n+1}$$

E , k , and θ_{TD} play different roles. E characterizes how fast, or how energetic, a given running gait is – a property of the apex state a gait starts from (Section 2.6), not something swept. k and θ_{TD} are the only two free parameters of the model itself (Section 2.3): there is no controller anywhere in this system, so a periodic gait at a given E is whatever P happens to produce for a chosen (k, θ_{TD}) , not something steered toward stability.

A periodic (self-repeating) gait is a fixed point of P : a height y^* such that

$$P(y^*) = y^*$$

equivalently, a root of

$$f(y) = P(y) - y$$

Local stability of y^* is governed by the derivative of P at the fixed point, $P'(y^*)$, estimated by a central finite difference:

$$P'(y^*) \approx \frac{P(y^* + dy) - P(y^* - dy)}{2 dy}$$

$$|P'(y^*)| < 1 : \text{stable (perturbations shrink hop over hop)}$$

$$|P'(y^*)| > 1 : \text{unstable (perturbations grow)}$$

The sign of $P'(y^*)$ carries meaning beyond the magnitude: $P'(y^*) < 0$ means a perturbation's sign flips every hop – oscillatory divergence or convergence, alternating above and below y^* – while $P'(y^*) > 0$ means a perturbation's sign is preserved every hop – monotonic divergence or convergence, drifting away from or toward y^* on one side. Section 4.3 reports the root-finding methodology and the result for the validated parameter set of Section 4.1.

3 Discussion of Assumptions

Several modeling choices were made to keep the SLIP model tractable with the tools of this course, each with a corresponding limitation:

- **Planar motion.** The model is restricted to the sagittal plane. Real running involves some lateral and rotational motion of the trunk and legs, which this model cannot capture.
- **Point mass, massless leg.** All body mass is lumped at a single point, and the leg itself carries no mass or rotational inertia. This isolates the center-of-mass dynamics that SLIP is meant to describe, but discards any effect of leg-swing inertia or trunk pitch, which the templates-and-anchors view [3] explicitly treats as detail that a template is allowed to omit.
- **Linear (Hookean) leg spring.** The stance leg is modeled as a single ideal linear spring, $F = -k(r - r_0)$, with constant stiffness k . Biological and robotic legs have leg stiffness that is nonlinear and can be actively modulated by the neuromuscular system or a controller; treating k as a fixed parameter to be swept is a simplification, not a claim that real legs behave this way.

- **Fixed touchdown angle held through flight.** θ_{TD} is assumed constant for the entire flight phase, i.e. there is no swing-leg retraction or angle adjustment in response to a disturbance before the foot lands. This is the standard SLIP assumption, but it means the model's only "control input" between hops is the touchdown angle chosen before each flight, not a continuous correction during flight.
- **Instantaneous, rigid, non-slipping foot placement.** Touchdown and liftoff are treated as instantaneous events with no impact losses, and the foot is assumed to stick to the ground (no slipping, no ground compliance) for the duration of stance at a fixed height $y = 0$. Real contact involves finite compliance and a friction cone that could, in principle, be violated at high touchdown angles or high speeds.
- **No stance-phase dissipation, no aerodynamic drag.** The stance-phase Lagrangian has no damping term, so mechanical energy is exactly conserved within a single stance phase (verified numerically in Section 4); flight is likewise treated as drag-free projectile motion. This was a deliberate choice: it isolates the elastic dynamics of interest and gives a clean, near-machine-precision check on the numerical integrator, at the cost of neglecting the fact that any real leg or running surface dissipates some energy each stride.
- **Equal rest length at touchdown and liftoff.** The leg is assumed to return to the same natural length r_0 between hops, with no active leg extension/retraction actuation changing that rest length from one stance phase to the next.

If time allows, discuss which of the assumptions above are most consequential for the stability results in Section 4 (e.g., does relaxing the zero-damping assumption change which (k, θ_{TD}) combinations are stable?), and note anywhere the numerical results below reveal a limitation of the model (such as touchdown angles/speeds that would violate a real friction cone).

4 Computational Results

4.1 Single-Stance Validation

The model is implemented in Python. `main.py` maps an apex state to the stance-phase initial condition via `touchdownState()`, then integrates `stance_RHS()` with `solve_ivp` (`rtol` = 10^{-10} , `atol` = 10^{-12}) to the liftoff event $r = r_0$, $\dot{r} > 0$, checking the conserved energy from Section 2.4 at every step. Using the parameters below, this validates the stance-phase model end to end:

m	r_0	k	g	θ_{TD}	y_0	v_{x0}
80 kg	1 m	20000 N/m	9.81 m/s ²	68°	1.2 m	3.0 m/s

	r [m]	θ [deg]	$\dot{r}$ [m/s]	$\dot{\theta}$ [rad/s]
Touchdown	1.0000	112.00	−3.2689	−1.9149
Liftoff	1.0000	83.48	3.1937	−1.6886

Stance lasts 0.2094 s; the leg compresses to 0.7740 m (22.6% of r_0); relative energy drift is 6.46×10^{-13} , i.e. machine precision, confirming that the integrator and the equations of motion in Section 2.3 are consistent. Figure 2 plots leg length, angle, CoM trajectory, and energy drift for this run.

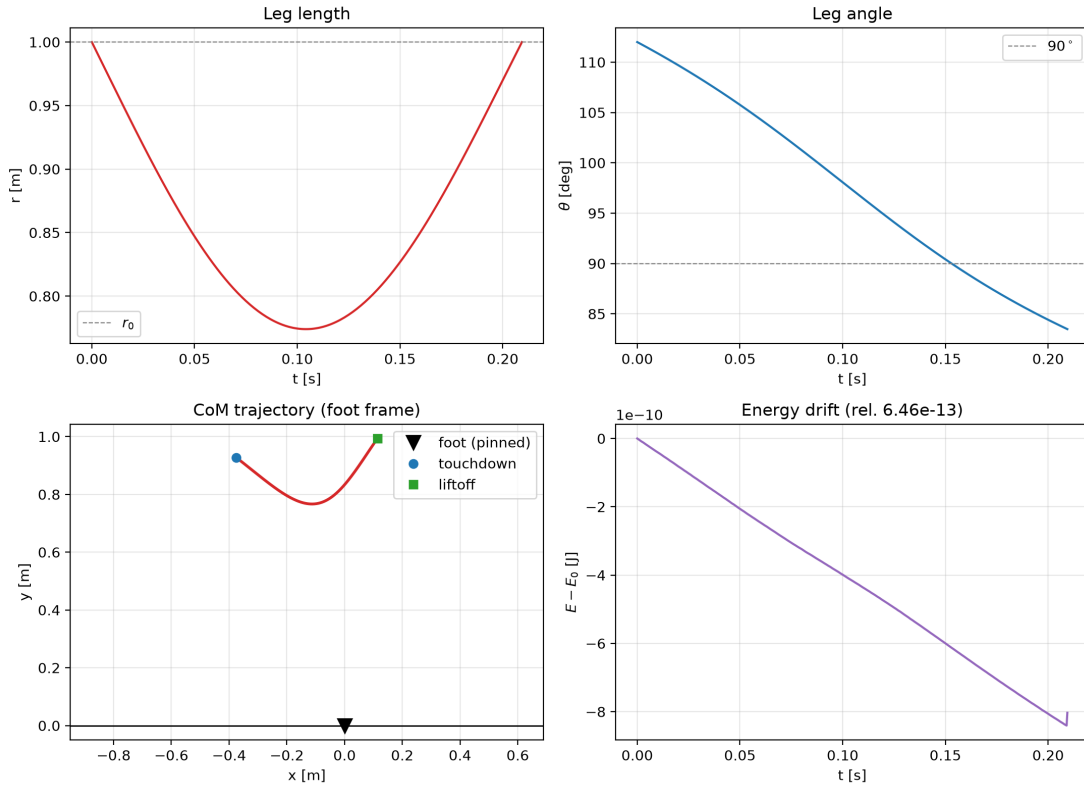


Figure 2: Stance simulation: $r(t)$, $\theta(t)$ (top), CoM trajectory (bottom left), energy drift (bottom right).

4.2 Full-Cycle Energy Conservation and Dimension-Reduction Validation

Beyond the single-stance check of Section 4.1, `proveX_dotIsRedundant()` validates the full hop cycle. Starting from the apex state of Section 4.1 ($y_0 = 1.2$ m, $v_{x0} = 3.0$ m/s), it computes the apex energy E_1 from the functional of Section 2.6 (`energyHop()`), chains one entire hop – flight → touchdown → stance → liftoff → flight (`hop()`) – to the next apex, and recomputes the apex energy E_2 there. Relative drift over the full cycle is

$$\frac{|E_2 - E_1|}{|E_1|} = 6.17 \times 10^{-13}$$

consistent with the stance-phase-only drift of Section 4.1 (6.46×10^{-13}), and confirming that total mechanical energy – not just the stance-phase quantity of Section 2.4 – is conserved across the full switching cycle.

The same run cross-checks the dimension reduction of Section 2.7. With E_1 fixed, the closed-form $\dot{x} = \sqrt{2E_1/m - 2gy}$ is evaluated at the simulated output apex height, $y_{n+1} = 1.4465$ m, predicting $\dot{x}_{pred} = 2.0403$ m/s; the simulation's own output horizontal velocity at that apex is $v_{x,n+1} = 2.0403$ m/s. Relative difference:

$$\frac{|v_{x,n+1} - \dot{x}_{pred}|}{|\dot{x}_{pred}|} = 2.41 \times 10^{-12}$$

i.e. the algebraic reduction of Section 2.7 is not only formally correct, it is numerically consistent with the simulator to machine precision.

4.3 Apex Return Map: Root-Finding Methodology and Result

`findApexMapFixedPoint()` locates the root of $f(y) = P(y) - y$ (Section 2.8) over the domain of y that is physically valid for a given (k, θ_{TD}) and E . The domain is bounded below by

$$y_{min} = r_0 \sin \theta_{TD}$$

the minimum apex height geometrically consistent with a touchdown at angle θ_{TD} (Section 2.2: the foot cannot be below ground at touchdown, so $y_{TD} = r_0 \sin \theta_{TD} \leq y_0$). It is bounded above by the height at which the closed-form $\dot{x}$ of Section 2.7 would require a negative value under the square root,

$$\frac{2E}{m} - 2gy \geq 0 \implies y \leq y_{max} = \frac{E}{mg}$$

At the exact lower boundary $y = y_{min}$, floating-point roundoff in the touchdown free-fall relation $\dot{y}_{TD} = -\sqrt{2g(y_0 - r_0 \sin \theta_{TD})}$ (Section 2.2) can push the argument of that square root slightly negative even though it is analytically exactly zero; a small inward padding, $y_{min} \rightarrow y_{min} + \epsilon$ ($\epsilon = 10^{-3}$ m in the implementation), avoids this without altering the physics.

$f(y)$ is sampled coarsely across $[y_{min}, y_{max}]$ (30 samples in the implementation), and consecutive samples are checked for a sign change. Once a bracket $[y_i, y_{i+1}]$ with $\text{sign}(f(y_i)) \neq \text{sign}(f(y_{i+1}))$ is found, Brent's method refines it to a root y^* . The derivative $P'(y^*)$ is then estimated by the central difference of Section 2.8 with $dy = 10^{-3}$ m.

For the validated parameter set of Section 4.1 ($m = 80$ kg, $r_0 = 1$ m, $k = 20000$ N/m, $\theta_{TD} = 68^\circ$, with E fixed by the apex state $y_0 = 1.2$ m, $v_{x0} = 3.0$ m/s):

$$y^* \approx 1.5096 \text{ m}, \quad P'(y^*) \approx -2.76$$

$|P'(y^*)| > 1$: this fixed point is unstable. The negative sign further indicates oscillatory divergence – a perturbed apex height does not drift monotonically away from y^* , it overshoots on alternating sides of y^* , growing in magnitude each hop. This is a genuine finding of the analysis, not a discrepancy to be explained away: $\theta_{TD} = 68^\circ$ was the value the project's working assumption had treated as the symmetric touchdown/liftoff angle, with an informal expectation that such a symmetric sweep would correspond to a stable periodic gait. It does not: at this (k, θ_{TD}) , steady hopping at $y^* \approx 1.5096$ m is unstable.

4.4 Grid Sweep Methodology and Parameter Sweep

`gridSearchFixedPoints()` wraps the single-case pipeline of Section 4.3 into a function of (k, θ_{TD}) that returns $(y^*, P'(y^*))$, or a null result if `findApexMapFixedPoint()` finds no sign change in $f(y)$ across the valid domain for that pair – i.e. no periodic gait exists there at this E . Each grid cell is evaluated against its own deep copy of the base parameter dictionary, with only k overwritten per cell, so sweeping k across the grid never mutates the shared dictionary read by other cells or other stages of the pipeline.

The grid itself is, for now, a coarse 5×5 sweep: k from 5000 to 40000 N/m (soft to stiff) and θ_{TD} from 50° to 80° , with each cell classified as stable ($|P'(y^*)| < 1$), unstable ($|P'(y^*)| > 1$), or no-fixed-point-found. This coarse pass is intended to be refined near the stability boundary and visualized as a contour or pcolormesh figure over (k, θ_{TD}) , per the deliverable below. The physically motivated bounds under consideration for that refinement are a lower angle bound near $\theta_{TD} \approx 59^\circ$ – the friction-cone floor found separately (constraint-force analysis at $\mu = 0.6$), below which the required ground-reaction force cannot be sustained without the foot slipping – up through steep angles of attack, and a k range wide enough to span qualitatively soft to stiff leg behavior.

The single-case pipeline (Section 4.3) and the grid-sweep methodology above are implemented; what remains is populating this subsection with the actual sweep output:

- An iteration plot (e.g. y_{n+1} vs. y_n , or y_n vs. hop index n) showing convergence to a fixed point for at least one *stable* (k, θ_{TD}) combination – Section 4.3 shows the 68° , 20000 N/m case is unstable, so a stable case must be located elsewhere in the grid first.
- The refined (k, θ_{TD}) grid itself: a contour or pcolormesh of $P'(y^*)$, or a discrete stable/unstable/no-fixed-point classification map.
- A short interpretation: does stability match the qualitative stiffness–speed trend reported by McMahon and Cheng [2]? Given Section 4.3’s result, is $\theta_{TD} = 68^\circ$ representative of the grid, or an outlier?

5 Lessons Learned

Address all three of the following once the project is complete:

1. **Benefits and limitations of the chosen approach.** What did the Lagrangian/generalized-coordinates + event-based hybrid-integration approach make easy, and where did it break down or require extra care (e.g. the touchdown/liftoff coordinate conversions, or the accuracy demands of chaining many hops through the return map)?
2. **What was learned by modeling this system.** E.g. anything surprising about the stance dynamics, the sensitivity of the return map to (k, θ_{TD}) , or the practical challenges of event-based integration that weren’t obvious before doing this project.
3. **Which course learning outcomes this project demonstrates.** Connect explicitly back to specific MEGN505 learning outcomes (Lagrangian mechanics, generalized coordinates, constraints, stability of periodic motion via Poincaré maps, numerical simulation of dynamical systems, etc.).

References

- [1] R. Blickhan, “The spring-mass model for running and hopping,” *Journal of Biomechanics*, vol. 22, no. 11–12, pp. 1217–1227, 1989.
- [2] T. A. McMahon and G. C. Cheng, “The mechanics of running: how does stiffness couple with speed?,” *Journal of Biomechanics*, vol. 23, suppl. 1, pp. 65–78, 1990.
- [3] R. J. Full and D. E. Koditschek, “Templates and anchors: neuromechanical hypotheses of legged locomotion on land,” *Journal of Experimental Biology*, vol. 202, no. 23, pp. 3325–3332, 1999.
- [4] W. J. Schwind and D. E. Koditschek, “Approximating the stance map of a 2-DOF monoped runner,” *Journal of Nonlinear Science*, vol. 10, pp. 533–568, 2000.
- [5] U. Saranli, M. Buehler, and D. E. Koditschek, “RHex: a simple and highly mobile hexapod robot,” *International Journal of Robotics Research*, vol. 20, no. 7, pp. 616–631, 2001.
- [6] H. Geyer, A. Seyfarth, and R. Blickhan, “Compliant leg behaviour explains basic dynamics of walking and running,” *Proceedings of the Royal Society B*, vol. 273, no. 1603, pp. 2861–2867, 2006.

Appendix: Code

main.py – touchdown map, stance-phase RHS, and the single-stance simulation and validation plots discussed in Section 4.1.

```

1 import copy
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from scipy.integrate import solve_ivp
5 from scipy.optimize import least_squares
6 from scipy.optimize import brentq
7
8 # Paramerts
9 m = 80
10 r_o = 1
11 k = 20000
12 g = 9.81
13 thetaTD = np.radians(68)
14 y_o = 1.2
15 v_o = 3.0
16
17 params = {'m' : m , 'k' : k , 'g' : g , 'r_o' : r_o , 'y_o' : y_o}
18
19
20 # =====
21 # Stage 1 - Stance phase: touchdown -> standing -> liftoff
22 # First piece of the model: integrate a single stance phase and
23 # check that total mechanical energy is conserved through it.
24 # =====
25
26 def touchdownState(x_o_dot , y_o , theta_TD , r_o , g):
27     r = r_o
28     theta = np.arctan2(r_o * np.sin(theta_TD) , -r_o * np.cos(theta_TD))
29
30     x_TD_dot = x_o_dot
31     y_TD_dot = - (2*g*(y_o - r_o * np.sin(theta_TD)))*.5
32
33     r_dot = x_TD_dot * np.cos(theta) + y_TD_dot * np.sin(theta)
34
35     theta_dot = (-x_TD_dot * np.sin(theta) + y_TD_dot * np.cos(theta)) / r_o

```

```

36
37     return [r , theta, r_dot , theta_dot]
38
39 def standingState(t , s, params):
40     r_dot = s[2]
41     theta_dot = s[3]
42
43     r_ddot = s[0] * theta_dot**2 - params['g'] * np.sin(s[1]) - (params['k'] /
44         params['m']) * (s[0] - params['r_o'])
45     theta_ddot = -(2/s[0]) * r_dot * theta_dot - (params['g'] / s[0]) * np.cos
46         (s[1])
47
48     return [r_dot , theta_dot, r_ddot, theta_ddot]
49
50 def liftoffStateSwitch(t, s, params):
51     return params['k'] * (s[0] - params['r_o'])
52 liftoffStateSwitch.terminal = True
53 liftoffStateSwitch.direction = 1
54
55 def energy(s, params):
56     E = (1/2) * params['m'] * (s[2]**2 + s[0]**2 * s[3]**2) + params['m'] *
57         params['g'] * s[0] * np.sin(s[1]) + (1/2) * params['k'] * (s[0] -
58         params['r_o'])**2
59
60     return E
61
62 # ----- run a single stance phase for diagnostics -----
63 # (mirrors the body of hop(), but keeps sol/s0 around for plotting)
64
65 def initialFunctionTest():
66     s0 = touchdownState(v_o, y_o, thetaTD, r_o, g)
67
68     t_span = (0.0, 1.0)
69     sol = solve_ivp(standingState, t_span, s0, args=(params,),
70         events=liftoffStateSwitch, rtol=1e-10, atol=1e-12,
71         dense_output=True, max_step=1e-3)
72
73     E = np.array([energy(sol.y[:, i], params) for i in range(sol.y.shape[1])])
74     drift = np.max(np.abs(E - E[0])) / np.abs(E[0])
75
76     # Plotting and data stuff, AI can fill in these gaps as simply learning to
77     # plot good is not hard and not my current goal
78
79     print("status:", sol.status)
80     print("liftoffStateSwitch time:", sol.t_events[0])
81     print("drift:", drift)
82
83     # ----- diagnostics -----
84     r_TD, th_TD, rdot_TD, thdot_TD = s0
85     s_LO = sol.y_events[0][0]
86     t_LO = sol.t_events[0][0]
87
88     print("--- touchdown state ---")
89     print(f" r      = {r_TD:.6f} m")
90     print(f" theta  = {np.degrees(th_TD):.4f} deg")
91     print(f" r_dot   = {rdot_TD:.6f} m/s")

```

```

87     print(f"   th_dot = {thdot_TD:.6f} rad/s")
88
89     print("--- liftoffStateSwitch state ---")
90     print(f"   status           = {sol.status}")
91     print(f"   stance duration= {t_L0:.6f} s")
92     print(f"   r                   = {s_L0[0]:.6f} m")
93     print(f"   theta              = {np.degrees(s_L0[1]):.4f} deg")
94     print(f"   r_dot              = {s_L0[2]:.6f} m/s")
95     print(f"   th_dot             = {s_L0[3]:.6f} rad/s")
96
97     print("--- validation ---")
98     print(f"   min leg length = {np.min(sol.y[0]):.6f} m")
99     print(f"   max compression= {params['r_o'] - np.min(sol.y[0]):.6f} m")
100    print(f"   E_0             = {E[0]:.6f} J")
101    print(f"   relative drift = {drift:.3e}")
102
103    # ----- figures -----
104    t = sol.t
105    r = sol.y[0]
106    th = sol.y[1]
107    x_com = r * np.cos(th)
108    y_com = r * np.sin(th)
109
110    fig, ax = plt.subplots(2, 2, figsize=(11, 8))
111
112    ax[0, 0].plot(t, r, color='C3')
113    ax[0, 0].axhline(params['r_o'], ls='--', lw=0.8, color='gray', label=
        '$r_o$')
114    ax[0, 0].set_xlabel('t [s]')
115    ax[0, 0].set_ylabel('r [m]')
116    ax[0, 0].set_title('Leg length')
117    ax[0, 0].legend()
118
119    ax[0, 1].plot(t, np.degrees(th), color='C0')
120    ax[0, 1].axhline(90, ls='--', lw=0.8, color='gray', label='$90^\circ$')
121    ax[0, 1].set_xlabel('t [s]')
122    ax[0, 1].set_ylabel(r'$\theta$ [deg]')
123    ax[0, 1].set_title('Leg angle')
124    ax[0, 1].legend()
125
126    ax[1, 0].plot(x_com, y_com, color='C3', lw=2)
127    ax[1, 0].plot(0, 0, 'kv', ms=10, label='foot (pinned)')
128    ax[1, 0].plot(x_com[0], y_com[0], 'o', color='C0', label='touchdown')
129    ax[1, 0].plot(x_com[-1], y_com[-1], 's', color='C2', label='
        liftoffStateSwitch')
130    ax[1, 0].axhline(0, color='k', lw=1)
131    ax[1, 0].set_xlabel('x [m]')
132    ax[1, 0].set_ylabel('y [m]')
133    ax[1, 0].set_title('CoM trajectory (foot frame)')
134    ax[1, 0].axis('equal')
135    ax[1, 0].legend()
136
137    ax[1, 1].plot(t, E - E[0], color='C4')
138    ax[1, 1].set_xlabel('t [s]')
139    ax[1, 1].set_ylabel('$E - E_0$ [J]')
140    ax[1, 1].set_title(f'Energy drift (rel. {drift:.2e})')

```

```

141     ax[1, 1].ticklabel_format(axis='y', style='sci', scilimits=(0, 0))
142
143     for a in ax.flat:
144         a.grid(alpha=0.3)
145     fig.tight_layout()
146     fig.savefig('stance_validation.png', dpi=200)
147     plt.show()
148
149
150 # =====
151 # Stage 2 - Apex state: ballistic flight after liftoff
152 # Convert the liftoff (r, theta) state back to Cartesian (x, y)
153 # and propagate the free-flight trajectory forward to the apex
154 # (the point where the vertical velocity goes to zero).
155 # =====
156
157 def projectileApexState(s):
158     r_o = s[0]
159     theta_o = s[1]
160     r_o_dot = s[2]
161     theta_o_dot = s[3]
162
163     # convert state from liftoff back to cartesian
164     y_o = r_o * np.sin(theta_o)
165     x_o = r_o * np.cos(theta_o)
166
167     y_o_dot = r_o_dot * np.sin(theta_o) + theta_o_dot * r_o * np.cos(theta_o)
168     x_o_dot = r_o_dot * np.cos(theta_o) - theta_o_dot * r_o * np.sin(theta_o)
169
170     # Solve for when y velocity = 0, this is the highest point
171     t_apex = y_o_dot / g
172
173     # find corresponding x y x_dot y_dot values
174     y_dot = -g*t_apex + y_o_dot
175     x_dot = x_o_dot
176
177     y = y_o - ((g/2) * t_apex**2) + y_o_dot * t_apex
178     x = x_o + x_dot * t_apex
179
180     return [x , y, x_dot , y_dot]
181
182
183 # =====
184 # Stage 3 - Hop: touchdown -> stance -> liftoff -> apex
185 # Chains stages 1-2 together into a single map that takes an
186 # apex state and returns the next apex state, one hop later.
187 # =====
188
189 def hop(apexState ,thetaTD, params):
190     s0 = touchdownState(apexState[2],apexState[1], thetaTD, params['r_o'],
191                          params['g'])
192     t_span = (0.0,1.0)
193
194     sol = solve_ivp(standingState, t_span, s0, args=(params,),
195                     events=liftoffStateSwitch, rtol=1e-10, atol=1e-12,
196                     dense_output=True, max_step=1e-3)

```

```

196     s1 = projectileApexState(sol.y_events[0][0])
197
198     return s1
199
200
201 def energyHop(apexState, params):
202     E = (1/2) * params['m'] * (apexState[2]**2 + apexState[3]**2) + params['m',
203         ] * params['g'] * apexState[1]
204     return E
205
206 # =====
207 # Stage 4 - x is redundant: the apex-to-apex map is fully
208 # determined by total energy E and height y, not by x directly.
209 # =====
210
211 def proveX_dotIsRedundant():
212     # E = (1/2)*m*(x_dot^2 + y_dot^2) + m*g*y
213     # ((E - mgy) * 2/m - y_dot**2)**.5 = x_dot
214     # Y_dot = 0
215     # ((E-mgy)*2/m)**.5 = x_dot
216     # (2E/m - 2gy)**.5 = x_dot
217
218     initialState = [0,y_o,v_o,0]
219     initialThetaTD = thetaTD
220
221     E1 = energyHop(initialState , params)
222
223
224     finalState = hop(initialState, initialThetaTD, params)
225     x_dot_pred = (2*E1/ params['m'] - 2*params['g']*finalState[1])**.5
226     E2 = energyHop(finalState, params)
227
228     print(f"Energy Drift over 1 hop: {E2-E1}")
229     print(f"X_dot Drift over 1 hop: {finalState[2]-x_dot_pred}")
230
231 # =====
232 # Stage 5 - Reduced 1-D apex map: fixed point and stability
233 # Stage 4 showed the hop only depends on E and y, so collapse the
234 # hop into a 1-D map y -> apexmap(y) and find the periodic apex
235 # height (fixed point), then check its local stability from the
236 # map's slope there.
237 # =====
238
239 def apexmap(y , thetaTD, params, E):
240     y = np.asarray(y).item()
241
242     x_dot = (2*E/ params['m'] - 2*params['g']*y)**.5
243
244     inputApexState = [0, y , x_dot , 0]
245     outputApexState = hop(inputApexState, thetaTD, params)
246
247     return outputApexState[1]
248
249 # ----- find the fixed point of the apex map and its local stability
250 # -----

```

```

250
251 def findApexMapFixedPoint(params, thetaTD, y_o=y_o, v_o=v_o):
252     initialState = [0, y_o, v_o, 0]
253     E = energyHop(initialState, params)
254     f = lambda y: apexmap(y, thetaTD, params, E) - y
255
256     y_min = params['r_o'] * np.sin(thetaTD) + .001 # apex must be at/above
257         touchdown height for this thetaTD
258     # 2E/m - 2*g*y >= 0
259     # Y < 2E / (2mg)
260     y_max = 2*E / (2* params['m'] * params['g'])
261
262     y_sample = np.linspace(y_min, y_max, 30)
263     f_sample = np.array([f(yi) for yi in y_sample])
264
265     sign_changes = np.where(np.diff(np.sign(f_sample)) != 0)[0]
266
267     if sign_changes.size == 0:
268         raise ValueError(f"no sign change in apex map for thetaTD={thetaTD}, k
269             ={params['k']}")
270
271     y_min = y_sample[sign_changes[0]]
272     y_max = y_sample[sign_changes[0] + 1]
273
274     y_star = brentq(f, y_min, y_max)
275     dy = .001
276
277     apexmapPrime = (apexmap(y_star + dy, thetaTD, params, E) - apexmap(y_star
278         - dy, thetaTD, params, E)) / (2 * dy) # type: ignore
279
280     # print(apexmapPrime)
281
282     return (y_star, apexmapPrime)
283
284 # =====
285 # Stage 6 - Grid search: fixed point and stability over (k, thetaTD)
286 # Each grid point needs its own params dict (deep-copied off the
287 # base params) so sweeping 'k' never mutates the shared dict that
288 # other stages/functions read from.
289 # =====
290
291 def gridSearchFixedPoints(base_params, k_values, thetaTD_values, y_o=y_o, v_o=
292     v_o):
293     y_star_grid = np.full((len(k_values), len(thetaTD_values)), np.nan)
294     slope_grid = np.full((len(k_values), len(thetaTD_values)), np.nan)
295
296     for i, k_val in enumerate(k_values):
297         for j, theta_val in enumerate(thetaTD_values):
298             trial_params = copy.deepcopy(base_params)
299             trial_params['k'] = k_val
300             try:
301                 y_star, slope = findApexMapFixedPoint(trial_params, theta_val,
302                     y_o, v_o)
303             except ValueError:
304                 continue

```



```

301         y_star_grid[i, j] = y_star
302         slope_grid[i, j] = slope
303
304     return y_star_grid, slope_grid
305
306
307 # =====
308 # Run
309 # =====
310
311 if __name__ == "__main__":
312     #     initialFunctionTest()
313     #     proveX_dotIsRedundant()
314     print(findApexMapFixedPoint(params, thetaTD))
315
316     k_sweep = np.linspace(5000, 40000, 5)
317     thetaTD_sweep = np.linspace(np.radians(50), np.radians(80), 5)
318     y_star_grid, slope_grid = gridSearchFixedPoints(params, k_sweep,
319                                                     thetaTD_sweep)
319     print("y_star grid:\n", y_star_grid)
320     print("slope grid:\n", slope_grid)

```

Once written, add the liftoff-map and apex-return-map / parameter sweep script here (e.g. `return_map.py`) with a matching `lstinputlisting` entry, alongside a short note describing what it computes.